НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені Ігоря СІКОРСЬКОГО»
Навчально-науковий фізико-технічний інститут
Кафедра математичних методів захисту інформації

Звіт до лабораторної №1

за темою:
Дослідження реалізації
протоколу SSL/TLS

Оформлення звіту:
Юрчук Олексій, ФІ-52мн

25 жовтня 2026 р.
м. Київ

ЗМІСТ

1 Мета і програмні засоби **1**
- 1.1 Мета . . . 1
- 1.2 Програмні засоби . . . 1

2 TLS Connection Research **1**
- 2.1 Параметри сертифікатів та самого з'єднання . . . 2
- 2.2 Subject Alternative Name (SAN) . . . 3
- 2.3 Сертифікатний ланцюг (Certificate Chain) . . . 4
- 2.4 TLS "рукостискання"(Handshake Process) . . . 4

3 Аналіз сучасних механізмів протоколу TLS **5**
- 3.1 Ключові відмінності між TLS 1.3 та TLS 1.2 . . . 5
- 3.2 Чому зараз вже не можна використовувати SSL та ранні версії TLS? . . . 5
- 3.3 Forward Secrecy і як вона захищає від компроментації ключа . . . 6
- 3.4 Certificate Transparency (прозорість сертифікатів) . . . 6
- 3.5 CRL та OCSP для перевірки скасування сертифікатів . . . 6
- 3.6 HTTP Strict Transport Security (HSTS) . . . 7
- 3.7 Необхідність перевірки попри шифрування . . . 8

4 Вразливості від атаки POODLE **8**
- 4.1 Заг.огляд . . . 8
- 4.2 Технічна складова . . . 8
- 4.3 Умови для успішної атаки . . . 9
- 4.4 Наслідки атаки . . . 10
- 4.5 Заходи щодо зменшення ризику та усунення вразливості . . . 10
- 4.6 Зв'язок з TLS 1.3 . . . 10

5 Аналіз сценаріїв порушення безпеки **10**

6 Висновки **13**

Список літературних і не дуже джерел **14**

1 Мета і програмні засоби

1.1 Мета

Метою цієї лабораторної роботи є дослідження практичної реалізації криптографічних механізмів у протоколі Transport Layer Security (TLS), вивчення принципів встановлення безпечного з'єднання, розгляд використання цифрових сертифікатів та аналіз заходів захисту від атак на безпечний канал зв'язку.

Зокрема, я для себе викоремив наступні завдання:

1. Практичне дослідження з'єднань TLS із мінімум двома (я обрав 3) загальнодоступними веб-сайтами
2. Аналіз сучасних механізмів безпеки TLS за допомогою аналізу низки технічних характеристик.
3. Поглиблене дослідження уразливості TLS через атаку POODLE (CVE-2014-3566).
4. Аналіз сценарію порушення безпеки, пов'язаного з вимкненою перевіркою сертифікатів.

1.2 Програмні засоби

Для дослідження з'єднань TLS, я на свому комп'ютері з ОС Windows 10 використовував наступні інструменти:

- **Web Browser (Google Chrome)** – вбудований переглядач сертифікатів (F12 → вкладка «Безпека») дозволяє перевірити сертифікат сервера, його ланцюжок, набір шифрів та версію TLS, узгоджену для поточного HTTPS-з'єднання.
- **OpenSSL** (через Powershell (можна і в Bash)) – утиліта командного рядка openssl `openssl s_client` підключається до віддаленого сервера та відображає повний стенограму TLS-рукостискання, узгоджений набір шифрів, ланцюжок сертифікатів та деталі сертифіката [1]. Приклад використання:

```
openssl s_client -connect google.com:443 -servername google.com
openssl s_client -connect google.com:443 -servername google.com | openssl
↪ x509 -noout -text
```

Додатково, ще знайшов онлайн-ресурс для перевірки TLS конфігурації:

- **SSL Labs Server Test** (https://www.ssllabs.com/ssltest/) – онлайн-інструмент, що виконує комплексний аналіз конфігурації TLS сервера, підтримуваних протоколів, наборів шифрування та відомих вразливостей.

2 TLS Connection Research

Для свого дослідження я обрав три загальнодоступні веб-сайти, розташовані в США:

1. **google.com** — глобальний постачальник технологій та найбільша пошукова система;
2. **amazon.com** — чи ненайбільша комерційна платформа;
3. **nist.gov** — Національний інститут стандартів і технологій США, урядовий ресурс.

Цим вибором я хотів забезпечити різноманітність у різних секторах (технології, електронна комерція, уряд) та різних центрах сертифікації.

2.1 Параметри сертифікатів та самого з’єднання

У таблицях 1 і 2 я зібрав підсумкові дані щодо параметрів з’єднання TLS та сертифікатів, отриманих для кожного веб-ресурсу.

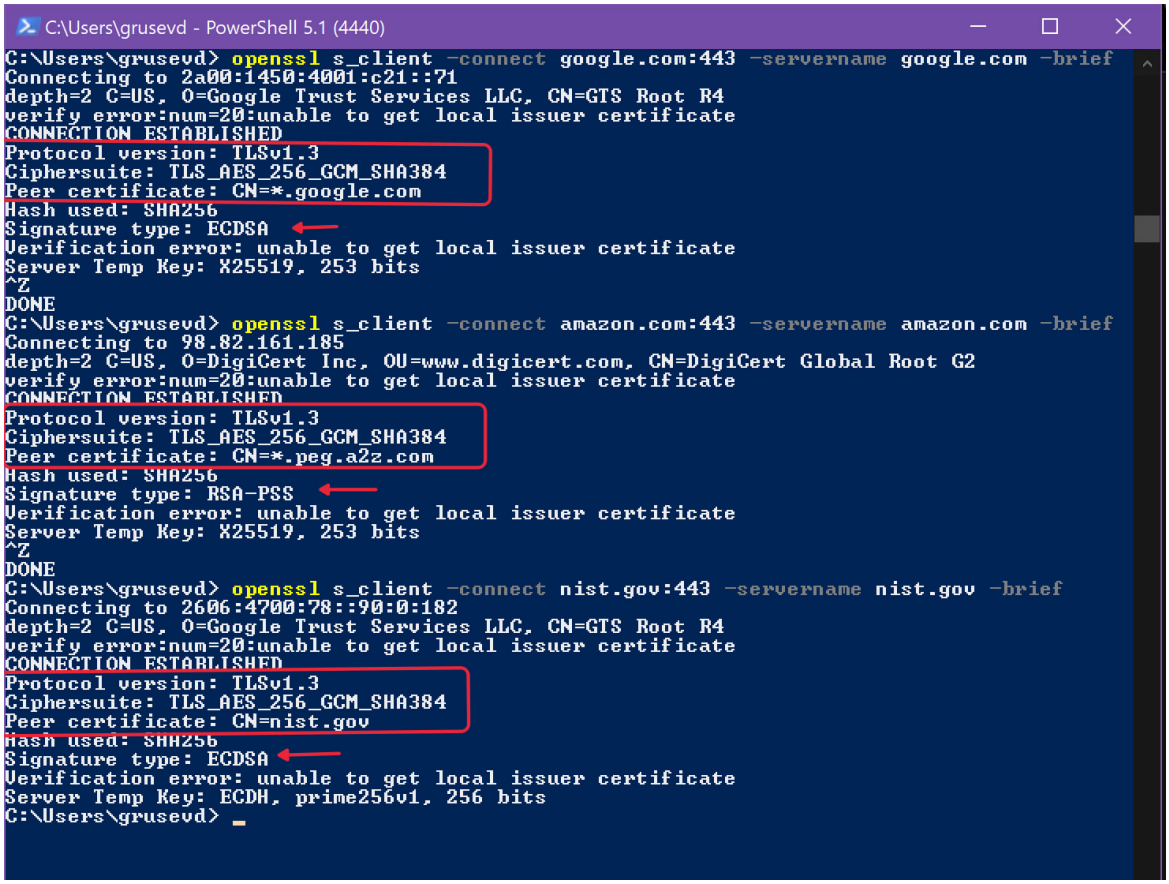


Рис. 1: Вивід команди OpenSSL `s_client` для google.com, amazon.com та nist.gov

Параметр	google.com	amazon.com	nist.gov
Версія TLS	TLS 1.3	TLS 1.3	TLS 1.2 / 1.3
Набір шифрів	TLS_AES_256_GCM_SHA384	TLS_AES_128_GCM_SHA256	TLS_AES_256_GCM_SHA384
Обмін ключами	X25519 (ECDHE)	X25519 (ECDHE)	ECDHE
Forward Secrecy	Так	Так	Так
HSTS	Так	Так	Так
Прозорість Сертифікатів	Так (SCTs)	Так (SCTs)	Так (SCTs)

Таблиця 1: Параметри TLS-з’єднання для досліджуваних веб-сайтів

Таблиця 2: Параметри цифрових сертифікатів для досліджуваних веб-сайтів

Параметр	**google.com**	**amazon.com**	**nist.gov**
Ім’я суб’єкта (CN)	*.google.com	www.amazon.com	*.nist.gov
Емітент (Issuer)	Google Trust Services (WR2)	Amazon RSA 2048 M04	DigiCert TLS RSA SHA256 2020 CA1
Public Key Algorithm	ECDSA P-256	RSA 2048-bit	RSA 2048-bit
Алгоритм підпису	ECDSA with SHA-256	SHA-256 with RSA	SHA-256 with RSA
Термін дії	Sep 28, 2026 – Dec 21, 2026 ∼3 months	Jan 19, 2026 – Jan 12, 2027 ∼12 months	Oct 22, 2026 – Jan 20, 2027 ∼3 months
Використання ключа	Digital Signature	Digital Signature, Key Encipherment	Digital Signature, Key Encipherment
SAN (selected)	*.google.com, .youtube.com, google.com, ...	www.amazon.com, amazon.com, .pci.amazon.com	*.nist.gov, nist.gov

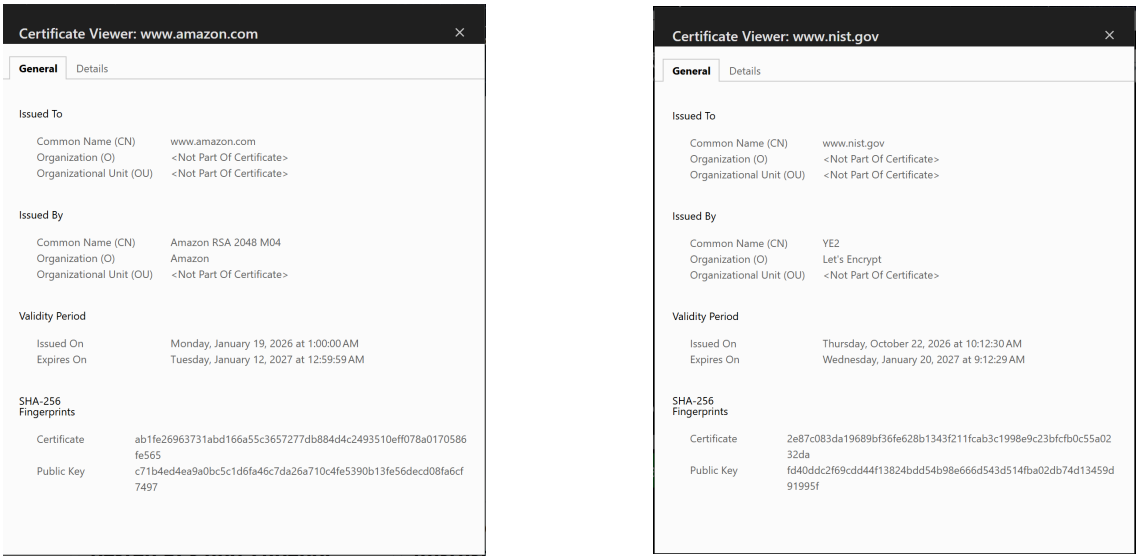


Рис. 2: Chrome certificate viewer for amazon.com. & nist.gov

2.2 Subject Alternative Name (SAN)

Розширення Subject Alternative Name «Альтернативна назва суб’єкта» (скорочено SAN), визначене в RFC 5280 [2], дозволяє одному сертифікату захищати кілька доменних імен. Наприклад, той же сертифікат Google охоплює понад 100 SAN, включаючи `*.google.com`, `*.youtube.com`, `*.googleapis.com`, та власне `google.com`. Це замінює старий підхід Common Name (CN). SAN є основним механізмом перевірки імені хоста, який використовують сучасні браузери. Якщо у полі ”Додаткові імена підписувача” (SAN entry) відсутній відповідний запис, браузер відхилить сертифікат, незважаючи на те, що CN збігається із запитуваним ім’ям хоста.

2.3 Сертифікатний ланцюг (Certificate Chain)

Кожен досліджуваний веб-сайт має власний ланцюжок з сертифікатів, який встановлює довіру від сертифіката сервера до кореневого центру сертифікації (root CA), якому довіряє операційна система або браузер:

- **google.com**: Сертифікат сервера (*.google.com) → GTS CA 2A3 (проміжний, Google Trust Services) → GTS Root R1 (root CA) [3].
- **amazon.com**: Сертифікат сервера (www.amazon.com) → Amazon RSA 2048 M04 (проміжний) → Amazon Root CA 1 (root CA) [4].
- **nist.gov**: Сертифікат сервера (*.nist.gov) → DigiCert TLS RSA SHA256 2020 CA1 (проміжний) → DigiCert Global Root CA (root CA).

Проміжні центри сертифікації мають взаємне підписання або попередньо встановлені зв'язки в сховищах довіри, що дозволяє клієнтам формувати ланцюг довіри без необхідності безпосереднього знання кожного CA-емітента. Ця ієрархічна модель, визначена в RFC 5280 [2], є основою web-PKI (Web Public Key Infrastructure).

2.4 TLS "рукостискання"(Handshake Process)

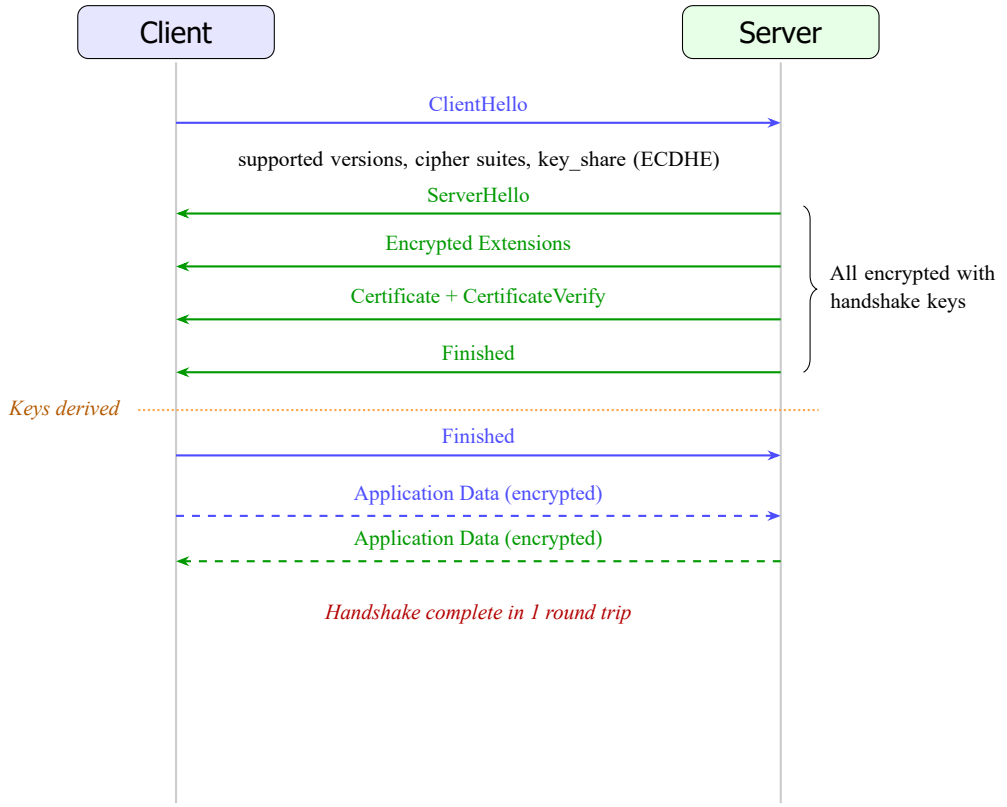


Рис. 3: Simplified TLS 1.3 handshake (1-RTT)

Взаємодія рукостискань TLS 1.3 [5], зображена на рисунку 3, забезпечує встановлення безпечного з'єднання за один цикл обміну даними (1-RTT), що є значним покращенням порівняно з двома циклами обміну даних, як це було в TLS 1.2. Цей процес забезпечує три ключові властивості безпеки:

1. **Server authentication**: Сервер підтверджує свою ідентичність, надаючи сертифікат, підписаний довіреним центром сертифікації (CA) генеруючи повідомлення `CertificateVerify`, яке підтверджує наявність відповідного приватного ключа.

2. **Cryptographic parameter negotiation**: Повідомлення `ClientHello` включає в себе набори шифрів та підтримувані способи обміну ключами. Сервер обирає найнадійніші з параметрів, що підтримуються обома сторонами, і відповідає, надіславши свою частку ключа.

3. **Key material derivation**: Обидві сторони здійснюють обмін ключами використовуючи алгоритм Elliptic Curve Diffie-Hellman Ephemeral (ECDHE). Зі спільного секрету за допомогою схеми виведення ключів на основі функції виведення ключів на основі HMAC (HMAC-based Key Derivation Function), виводяться окремі ключі для встановлення з'єднання та трафіку додатків. Також є важливим те, що використовуються тимчасові ключі, що забезпечує пряму секретність [6].

3 Аналіз сучасних механізмів протоколу TLS

3.1 Ключові відмінності між TLS 1.3 та TLS 1.2

TLS 1.3, станадартизований у RFC 8446 [5], вносить фундаментальні зміни порівняно з TLS 1.2 [7]. Тож, закинувши ці два документи в ШІ, я отримав наступне порівняння:

Змінена складова	**TLS 1.2**	**TLS 1.3**
Затримка з'єднання (latency)	2-RTT	1-RTT (+ 0-RTT resumption)
Набори шифрів	37+ (including insecure)	5 AEAD-only suites
Обмін ключами	RSA, DHE, ECDHE	ECDHE only (X25519, P-256)
Режими шифрування	CBC, stream, AEAD	AEAD only (GCM, ChaCha20)
Пряма секретність	Optional	Mandatory
Шифрування рукостискань	Plaintext	Encrypted after ServerHello
Відновлення	Session ID / ticket	PSK-based

Таблиця 3: Порівняння TLS 1.2 і TLS 1.3

Найбільш значущими змінами є примус до обов'язкового використання прямої секретності (forward secrecy), шляхом усунення статичного обміну ключами RSA (RSA key exchange), скорочення наборів шифрів до п'яти варіантів на основі AEAD та зашифрований обмін даними (handshakes), що приховують сертифікат сервера від пасивних зловмисників, які намагаються перехопити дані [5].

3.2 Чому зараз вже не можна використовувати SSL та ранні версії TLS?

SSL 2.0, SSL 3.0 [8], TLS 1.0, і TLS 1.1 містять фундаментальні криптографічні слабкості, які неможливо усунути за допомогою налаштувань:

- **SSL 3.0**: вразливий до атак типу POODLE (CVE-2014-3566), які використовують недетерміноване CBC padding [9]. У самому протоколі немає виправлення цього недоліку.
- **TLS 1.0**: вразливий до атаки BEAST (ланцюгове з'єднання векторів ініціалізації CBC) та використовує MD5-SHA-1 у своїй Pseudo-Random Function (PRF).
- **TLS 1.1**: Хоча в ньому виправлено вищезгадану проблему з ланцюгом векторів ініціалізації (IV), він все ще підтримує слабкі набори шифрів такі як RC4, 3DES, не вимагає шифрування AEAD, а його PRF базується на MD5 та SHA-1.

Стандарт RFC 8996 [10] офіційно визнає TLS 1.0 та 1.1 застарілими, а NIST SP 800-52 Rev. 2 [11] у свою чергу встановлює TLS 1.2 як мінімальний стандарт для федеральних систем США, при цьому наполегливо рекомендуючи використовувати TLS 1.3. Усі браузери з 2020 року відключили підтримку цих застарілих версій.

3.3 Forward Secrecy і як вона захищає від компроментації ключа

«Forward Secrecy» (також відома як «Perfect Forward Secrecy», PFS) – це властивість протоколів узгодження ключів, що гарантує, що сесійні ключі не можуть бути скомпрометовані, навіть якщо довгостроковий приватний ключ сервера згодом стане відомим [6].

На практиці це досягається за допомогою обміну ефемерними ключами за алгоритмом Діффі–Хеллмана (ephemeral Diffie-Hellman key exchange, DHE або ECDHE): кожна сесія TLS генерує нові, короткочасні пари ключів, які використовуються для обчислення спільного секрету, а потім одразу видаляються. Навіть якщо зловмисник запише весь зашифрований трафік і згодом отримає приватний ключ сервера, він не зможе відтворити ефемерні ключі і, отже, не зможе розшифрувати минулі сесії.

Без даної функції, як і у випадку зі статичним обміном ключами RSA у TLS 1.2, приватний ключ RSA сервера використовується безпосередньо для розшифрування pre-master secret. Якщо цей ключ буде скомпрометований внаслідок злому сервера або майбутніх відкриттів в криптоаналізі, то усі раніше записані сесії можна буде розшифрувати ретроактивно (заднім числом). TLS 1.3 повністю усуває цей ризик, обов'язково застосовуючи ECDHE для всіх з'єднань [5].

3.4 Certificate Transparency (прозорість сертифікатів)

Прозорість сертифікатів Certificate Transparency (CT), була визначена вперше у RFC 6962 [12], являє собою систему публічно перевірювальних журналів, до яких можна лише додавати записи, що фіксують кожен сертифікат, виданий учасником системи сертифікації. Її мета – виявляти неправильно або зловмисно видані сертифікатів.
Механізм роботи такий [13]:

1. Центр сертифікації видає сертифікат і надсилає його до одного або кількох журналів CT.
2. Журнал повертає підписаний часовий штамп сертифіката (Signed Certificate Timestamp, SCT) – криптографічне підтвердження того, що сертифікат буде включено до журналу протягом визначеного часового вікна (Maximum Merge Delay).
3. SCT вбудовується в сертифікат (через розширення X.509), передається через розширення TLS або надається за допомогою OCSP-степлінгу.
4. Браузери (зокрема Chrome з 2018 року) вимагають наявності дійсних SCT з декількох незалежних журналів, перш ніж довіряти цьому сертифікату.
5. Власники доменів та служби моніторингу постійно сканують журнали CT, щоб виявити несанкціоновані сертифікати для своїх доменів.

Certificate Transparency виявився вирішальним у виявленні інцидентів, зокрема несанкціонованих сертифікатів Google, що прогримів в 2015 році, виданих CNNIC, та скандалу з помилковою видачею сертифікатів Symantec у 2017 році. Наразі основні центри сертифікації за замовчуванням надсилають усі сертифікати до журналів CT [14].

3.5 CRL та OCSP для перевірки скасування сертифікатів

Коли сертифікат необхідно анулювати до закінчення його природного терміну дії (через компрометацію ключа, зміну політики ЦС або зміну власника домену), існують два механізми для повідомлення про статус анулювання [2, 15]:

Certificate Revocation Lists (CRL) [2]:

- Центр сертифікації періодично публікує підписаний список серійних номерів усіх анульованих сертифікатів.
- Клієнти завантажують собі весь список CRL і перевіряють, чи є в ньому серійний номер їхнього сертифіката.
- *Обмеження*: списки CRL досягають іноді значних розмірів, оскільки це мільйони записів для великих ЦС, оновлюються вони нечасто (від кількох годин до кількох днів), а їх завантаження спричиняє затримку. Анонімність клієнта збережена.

Online Certificate Status Protocol (OCSP) [15]:

- Клієнт надсилає запит у режимі реального часу до OCSP-відповідача ЦС із серійним номером сертифіката.
- Респондер повертає підписаний статус ”good”, ”revoked” or ”unknown”.
- *Обмеження*: OCSP створює проблему конфіденційності, оскільки ЦС дізнається, які сайти відвідує клієнт, збільшує затримку для кожного нового TLS з’єднання та створює єдину точку відмови (single failure point) – якщо респондер недоступний, клієнт повинен або перейти в режим ”fail-open” (незахищений), або в режим ”fail-closed (втрата доступності).

Технологія **OCSP Stapling** дозволяє усунути обидві проблеми: сервер періодично фетчить власну OCSP-відповідь і ”прикріплює” її до процесу встановлення з’єднання TLS, завдяки чому клієнту не потрібно звертатися до центру сертифікації [16]. Однак впровадження цієї технології залишається неповним, і ще не всі сервери її реалізують.

3.6 HTTP Strict Transport Security (HSTS)

HTTP Strict Transport Security (скорочено HSTS), визначений у стандарті RFC 6797 [17], є механізмом політики, за допомогою якого веб-сайт вказує браузерам, що взаємодіяти з ним необхідно виключно через HTTPS-з’єднання. Сервер повідомляє про цю політику через заголовок HTTP-відповіді `Strict-Transport-Security`:

```
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
```

HSTS захищає від таких атак як:

- **SSL stripping**: Зловмисник у мережі перехоплює початковий HTTP-запит і переписує всі HTTPS-посилання на HTTP, змушуючи жертву вважати, що сайт не підтримує шифрування. Якщо HSTS увімкнено, браузер відмовляється надсилати будь-які HTTP-запити на цей домен.
- **Protocol downgrade attacks**: Зловмисник намагається примусово змусити з’єднання використовувати HTTP замість HTTPS. HSTS запобігає цьому, обробляючи всі запити до домену виключно як HTTPS.
- **Cookie hijacking**: Без HSTS файли cookie, встановлені через HTTPS, можуть надсилатися через HTTP, якщо не встановлено прапорець `Secure`. HSTS примусово використовує HTTPS для всіх запитів, що зменшує цей вектор атаки.

Список попереднього завантаження HSTS (який підтримується у Chromium і використовується іншими браузерами) йде ще далі – домени зі списку жорстко задані як такі, що підтримують лише HTTPS, що усуває навіть вікно вразливості під час першого відвідування до отримання заголовка HSTS [17]. Усі три обрані мною веб-сайти (`google.com`, `amazon.com`, `nist.gov`) реалізують HSTS.

3.7 Необхідність перевірки попри шифрування

Шифрування саме по собі хоч і забезпечує конфіденційність даних під час передачі, але без перевірки сертифіката клієнт не має гарантії того, хто саме знаходиться на іншому кінці зашифрованого каналу. Це створює критичну вразливість! Це відбувається в такі етапи:

1. Зловмисник, який здійснюватиме атаку ”людина посередині” (Man-in-the-Middle, MITM), може пред’явити *власний* сертифікат і встановити з клієнтом цілком дійсний зашифрований сеанс TLS.
2. Одночасно з цим зловмисник встановлює окремий зашифрований сеанс із легітимним сервером.
3. Дані проходять через зловмисника у вигляді відкритого тексту між двома зашифрованими тунелями, надаючи йому повний доступ для читання та запису до всіх переданих даних

А перевірка сертифіката гарантує, що суб’єкт, який надає сертифікат, є законним ”працівником” (оператором) даного домену. Процес перевірки контролює:

1) чи підписаний сертифікат довіреним центром сертифікації;
2) чи збігається SAN сертифіката із запитуваним іменем хоста;
3) чи не закінчився термін дії сертифіката;
4) чи не було сертифікат відкликано.

Без цих перевірок в реальності шифрування створює хибне відчуття безпеки – канал шифрується, але сторона на іншому кінці може виявитися зловмисником. Цей варіант я детальніше розглянув разом з прикладом у розділі 5.

4 Вразливості від атаки POODLE

4.1 Заг.огляд

POODLE (Padding Oracle On Downgraded Legacy Encryption), вразливість у SSL 3.0, шо задокументована як CVE-2014-3566 [18], була відкрита співробітниками Google в жовтні 2014 [9]. Вона дозволяє активному мережевому зловмиснику розшифрувати окремі байти трафіку, зашифрованого за протоколом SSL 3.0 у Cipher Block Chaining (CBC) mode.

4.2 Технічна складова

Ця вразливість використовує фундаментальний конструктивний недолік у специфікації CBC padding (заповнення) у протоколі SSL 3.0 [8]:

- У протоколі SSL 3.0 режим CBC вимагає заповнення, щоб довести обсяг відкритого тексту до кратного розміру блоку (8 байтів для 3DES, 16 байтів для AES).
- Саме заповнення може складається з довільних байтів, після яких йде один (значущий для сервера) байт, що вказує на довжину заповнення. І перевіряється саме він. Вразливість протоколу полягає в саме в цьому. Наприклад паддінг довжини 3, останні три байти ”AA”, ”FF”, ”03”, первірятиметься останній.
- На противагу цьому, TLS 1.0+ вимагає, щоб усі байти заповнення мали те саме значення, що й байт довжини, а одержувач перевіряє кожен байт. Тобто в ідеалі, для уникнення атаки мало би бути ”03”, ”03”, ”03”. Більш детально на прикладі нижче.

Ця неповна перевірка створює так званий padding oracle (хз як перекласти), що дозволяє зловмиснику маніпулювати блоками шифрованого тексту та спостерігати, чи сервер приймає чи відхиляє змінений запис:

1. Зловмисник визначає (підбором) блок шифрованого тексту C_i, що містить цільовий байт для розшифрування.
2. Він замінює останній блок шифрованого тексту (який містить заповнення) на C_i.
3. Оскільки під час розшифрування сервер перевіряє лише останній байт розшифрованого блоку, то з ймовірністю $1/256$, останній байт виявляється дійсною довжиною заповнення (від 0x00 до розміру блоку мінус 1), і запис буде прийнято.
4. Коли запис приймається, зловмисник може отримати цей байт відкритого тексту за допомогою операції XOR:
$$P_i[\text{last}] = D(C_i)[\text{last}] \oplus C_{i-1}[\text{last}],$$
де $D(C_i)[\text{last}]$ відоме значення padding (заповнення). І так далі поки не розшифрує весь текст.

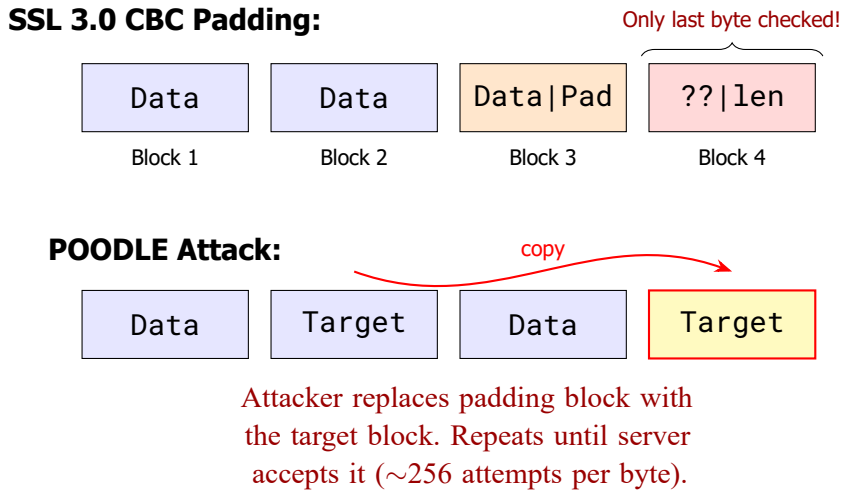


Рис. 4: POODLE attack: CBC padding oracle in SSL 3.0

4.3 Умови для успішної атаки

Атака POODLE вимагає виконання наступних умов [9, 19]:

1. **Активність «людини посередині» (Man-in-the-Middle):** зловмисник повинен мати можливість перехоплювати та модифікувати мережевий трафік між клієнтом і сервером (наприклад, у спільній мережі Wi-Fi).
2. **Підтримка SSL 3.0. як клієнтом, так і сервером:** навіть якщо обидві сторони підтримують TLS 1.2, зловмисник може змусити здійснити пониження версії протоколу (пункт нижче).
3. **Механізм пониження версії протоколу:** зловмисник порушує спроби встановлення з'єднання TLS, змушуючи клієнта перейти SSL 3.0. Цей "фінт вухами", так званий, "downgrade dance", використовує властивість браузерів зворотної сумісності.
4. **Можливість ініціювати повторювані запити:** Зловмиснику потрібно, щоб браузер жертви надсилав багато запитів до цільового сервера; зазвичай це досягається шляхом використання JavaScript через HTTP на іншій сторінці (на сусідньому HTTP-сайті або в рекламній мережі).

5. **Набір шифрів CBC:** З'єднання має використовувати набір шифрів у режимі CBC (який SSL 3.0 підтримує виключно разом із RC4, що сам по собі є легкокомпроментованим і вразливим)

4.4 Наслідки атаки

Успішна атака POODLE дозволяє розшифрувати конфіденційні дані по одному байту зі швидкістю приблизно 256 запитів на кожен розшифрований байт. Основними цілями такої атаки виявляються:

- Сесійні файли cookie HTTP, що дозволяють перехопити сесію;
- Токени автентифікації та токени CSRF;
- Частини заголовків або тіл запитів, що містять конфіденційну інформацію.

Хоч ця атака і є відносно повільною порівняно з масовим розшифруванням, та для 16-байтового сесійного файлу cookie потрібно лише приблизно 4 096 запитів, що є здійсненним за лічені хвилини за сприятливих умов мережі [9].

4.5 Заходи щодо зменшення ризику та усунення вразливості

Вище зазначені вразливості було усунуто за допомогою кількох підходів [20, 21]:

1. **Вимкнення SSL 3.0**: Усі сучасні браузери та сервери остаточно вимкнули підтримку SSL 3.0.
2. **TLS_FALLBACK_SCSV** (RFC 7507) [20]: Механізм сигналізації, за якого клієнт має право включити спеціальне значення у набір шифрів для пониження версії. Якщо сервер бачить це значення, але підтримує вищу версію протоколу, він відхиляє з'єднання, запобігаючи потенційно можливій атаці.
3. **Серверні налаштування**: Всім адміністраторам було рекомендовано негайно вимкнути SSL 3.0 на веб-серверах (Apache: `SSLProtocol All -SSLv3`; Nginx: `ssl_protocols TLSv1.2 TLSv1.3`), але не всі могли дослухатися :)

4.6 Зв'язок з TLS 1.3

POODLE **не є застосовним до TLS 1.3** [5]. TLS 1.3 усуває цю вразливість завдяки низці проектних рішень:

- Набори шифрів CBC повністю видалено; підтримуються лише AEAD (AES-GCM, ChaCha20-Poly1305), що забезпечують автентифікацію всього шифрованого тексту та не мають оракула заповнення (padding oracle).
- Механізм захисту від пониження версії вбудовано в протокол: сервер вбудовує контрольне значення у свою відповідь ServerHello, під час узгодження версії, нижчої за максимальну, що дозволяє клієнту виявляти ненавмисне пониження версії.
- SSL 3.0, TLS 1.0, та TLS 1.1 не входять до переліку версій, що узгоджуються в TLS 1.3's (Розширення `supported_versions` замінює застаріле поле версії).

5 Аналіз сценаріїв порушення безпеки

Ситуація: Розробник клієнтського застосунку під час встановлення TLS-з'єднання стикається з помилкою перевірки цифрового сертифіката сервера. Для усунення помилки він вимикає перевірку сертифіката. Після цього TLS-з'єднання успішно встановлюється, і дані між клієнтом і сервером передаються у зашифрованому вигляді.

Чому вимкнення перевірки дозволило встановити з'єднання?

Помилки перевірки сертифіката виникають, коли бібліотека TLS клієнта не може перевірити сертифікат сервера на відповідність власному сховищу довірених сертифікатів. До типових причин належать: сертифікат самопідписаний, виданим ненадійним або внутрішнім центром сертифікації, термін дії якого закінчився, або має невідповідність імені хосту (запитаний домен не збігається з жодним записом SAN) [2].

Вимкнувши перевірку, розробник дає вказівку бібліотеці TLS пропустити всі перевірки сертифікатів і прийняти *будь-який сертифікат*, наданий віддаленою стороною – незалежно від того, чи є він легітимним, самопідписаним, простроченим чи створеним зловмисником. Сам процес встановлення з'єднання TLS (обмін ключами, узгодження шифру, налаштування шифрування) відбувається нормально, оскільки ці етапи не залежать від перевірки сертифіката. З'єднання функціонує в тому сенсі, що дані передаються у зашифрованому вигляді за протоколом, але фундаментальне питання про те, *з ким саме* спілкується клієнт, залишається відкритим.

Які властивості безпеки зберігаються, а які втрачаються?

Властивість	Normal TLS	No Verify	Пояснення
Конфіденційність	Є	Часткова	Зашифровано, але можна буде зламано зловмисником
Цілісність	Є	Часткова	MAC-protected, але атака MITM може змінювати дані
Серверна автентифікація	Є	Відсутня	Жодного доказу, хто видає себе за сервер
Forward Secrecy	Є	Є	ECDHE ключі все ще є тимчасовими
Незаперечність	Часткова	Відсутня	Немає прив'язки до фіксованої ідентичності на іншому боці

Таблиця 4: Властивості безпеки з перевіркою сертифіката та без неї

При вимкненій перевірці сертифікатів **автентифікація сервера повністю втрачається**. Механізми шифрування та цілісності (симетричні шифри, MAC/AEAD-теги) все ще працюють коректно під час передачі даних, але вони захищають канал *лише* від пасивних підслуховувачів. У разі активного зловмисника, який здійснює атаку «людина посередині», порушуються все – як конфіденційність, так і цілісність, оскільки зловмисник розриває TLS-з'єднання з обох боків, отримуючи доступ до відкритого тексту між двома зашифрованими тунелями.

Чи може зловмисник читати або змінювати дані?

Є два варіанти зловмисника: перший, це пасивний зловмисник (лише прослуховування) – він може лише читати, дані в мережі все ще зашифровані за допомогою надійних алгоритмів, і без сесійних ключів не може розшифрувати трафік. А якщо зловмисник є активним, і здійснює MITM, то так, повністю. Зловмисник надає клієнту власний (підставний) сертифікат (який клієнт приймає без перевірки) та окремо підключається до легітимного сервера. Зловмисник розшифровує весь трафік від клієнта, читає/змінює його, повторно шифрує та пересилає на сервер. Той самий процес відбувається у зворотному напрямку для відповідей сервера. Зловмисник має повний доступ до всіх переданих даних у вигляді відкритого тексту.



Рис. 5: MITM attack, коли перевірка сертифікатів вимкнена

За яких умов можлива атака MITM?

Атака MITM стає можливою, коли зловмисник може розміститися на шляху мережі між клієнтом і сервером. За наступних умов:

- **Використання публічних мереж Wi-Fi:** Зловмисник, що перебуває в тій самій мережі, може використовувати підробку ARP-пакетів для перенаправлення трафіку через свій пристрій.
- **Зламана мережева інфраструктура:** Зловмисний або зламаний маршрутизатор, комутатор або DNS-сервер може перенаправляти трафік до лап зловмисника.
- **DNS spoofing or hijacking:** Зловмисник перенаправляє доменне ім'я на свою власну IP-адресу.
- **BGP hijacking:** На рівні інтернет-провайдера маніпуляції з маршрутами можуть перенаправити трафік, призначений для діапазону IP-адрес сервера.
- **Зламаний проксі-сервер або використання VPN:** Якщо клієнт маршрутизує трафік через проксі-сервер або VPN, що контролюється зловмисником, весь трафік стає доступним.

Якщо перевірка сертифікатів увімкнена, клієнт виявить сертифікат зловмисника і перерве з'єднання. Без неї клієнт сліпо довіряє будь-якому сертифікату, який йому надають.

Коректне вирішення проблеми з перевіркою сертифіката

Замість того, щоб вимкнути перевірку, розробник мав би виявити та усунути першопричину помилки сертифіката:

1. **Expired сертифікат:** Найбанальніше оновити сертифікат сервера у центрі сертифікації (CA).
2. **Невідповідність імені хосту (Hostname mismatch):** Переконатися, що SAN сертифіката містить точне ім'я хосту або шаблон із символом підстановки, який клієнт використовує для підключення.
3. **Self-signed або internal центр сертифікації:** Встановити кореневий сертифікат центру сертифікації у сховище довіри клієнтського додатка. (під час розробки це було б прийнятним, а у виробничому середовищі – ні. Треба було використовувати загальновизнаний центр сертифікації)
4. **Incomplete ланцюг сертифікатів:** Налаштувати сервер так, щоб він надсилав повний ланцюг (сертифікат сервера + проміжні CA). Відсутність проміжних сертифікатів вважається поширеною помилкою налаштування.
5. **Фіксація сертифіката (Certificate pinning):** Для додатків з високим рівнем безпеки треба було зафіксувати очікуваний сертифікат або відкритий ключ у клієнтському коді, забезпечивши додатковий рівень перевірки, що виходить за межі стандартної моделі довіри до ЦС.

Ні за яких обставин не слід вимикати перевірку сертифікатів у виробничому коді! Багато мов програмування надають зручні, але при цьому небезпечні можливості для цього (наприклад, Python's `verify=False`, Node.js's `rejectUnauthorized: false`, Java's `TrustAllCerts`, і т.п.), їх слід позначати під час огляду коду як критичні недоліки безпеки, з подальшим виправленням.

6 Висновки

Ця лабораторна робота стала для мене практичним дослідженням реалізацій протоколу TLS, поєднавши практичне вивчення активних HTTPS-з'єднань з аналізом базових криптографічних механізмів. Дослідження трьох веб-сайтів (google.com, amazon.com, nist.gov) продемонструвало, що сучасні сервери переважно використовують TLS 1.3 із наборами шифрів AEAD, обов'язковою прямим секретністю через ECDHE та прозорістю сертифікатів. Впровадження компанією Google сертифікатів ECDSA та коротких термінів дії (приблизно 90 днів) є підходом, орієнтованим на безпеку, тоді як Amazon продовжує використовувати RSA 2048 із довшим терміном дії.

Аналіз механізмів TLS підтвердив, що TLS 1.3 являє собою зміну парадигми в безпеці передачі даних: обов'язкова forward secrecy, зменшена latency, зашифровані повідомлення під час встановлення з'єднання та значно спрощений набір шифрів усувають цілі класи історичних вразливостей. Уразливість POODLE (CVE-2014-3566) продемонструвала, як, здавалося б, незначна помилка в проектуванні протоколу – недостатня специфікація padding proof CBC у SSL 3.0 створює реальний вектор атаки. Її дослідження підтверджує принцип, що розробка криптографічних протоколів вимагає точної специфікації та рішучої відмови від застарілих версій.

Сценарій перевірки сертифікатів продемонстрував, що шифрування без автентифікації створює лише ілюзію(!) безпеки. Безпека TLS вимагає обох елементів: шифрування для захисту даних під час передачі та перевірки сертифікатів, щоб гарантувати, що дані дійдуть до призначеного одержувача. Вимкнення перевірки в коді додатка є критичним дефектом безпеки, який уможливлює атаки типу "man-in-the-middle"(MITM).

Список літературних і не дуже джерел

[1] OpenSSL Software Foundation. *OpenSSL Documentation — s_client*. 2024. (URL).

[2] David Cooper et al. *RFC 5280 — Internet X.509 PKI Certificate and CRL Profile*. 2008. (URL) (visited on 10/21/2026).

[3] Google Trust Services. *Google Trust Services — Root CAs*. 2024. (URL) (visited on 10/20/2026).

[4] Amazon Trust Services. *Amazon Trust Services Repository*. 2024. (URL) (visited on 10/20/2026).

[5] Eric Rescorla. *RFC 8446 — The Transport Layer Security (TLS) Protocol Version 1.3*. 2018. (URL) (visited on 10/22/2026).

[6] Cloudflare. *What Is Forward Secrecy?* 2024. (URL) (visited on 10/22/2026).

[7] Tim Dierks and Eric Rescorla. *RFC 5246 — The Transport Layer Security (TLS) Protocol Version 1.2*. 2008. (URL) (visited on 10/22/2026).

[8] Alan Freier, Philip Karlton, and Paul Kocher. *RFC 6101 — The Secure Sockets Layer (SSL) Protocol Version 3.0*. 2011. (URL) (visited on 10/20/2026).

[9] Bodo Möller, Thai Duong, and Krzysztof Kotowicz. *This POODLE Bites: Exploiting The SSL 3.0 Fallback*. 2014. (URL) (visited on 10/24/2026).

[10] Kathleen Moriarty and Stephen Farrell. *RFC 8996 — Deprecating TLS 1.0 and TLS 1.1*. 2021. (URL) (visited on 10/20/2026).

[11] Kerry McKay and David Cooper. *NIST SP 800-52 Rev. 2 — Guidelines for TLS Implementations*. 2019. (URL) (visited on 10/24/2026).

[12] Ben Laurie, Adam Langley, and Emilia Kasper. *RFC 6962 — Certificate Transparency*. 2013. (URL) (visited on 10/21/2026).

[13] Google. *Certificate Transparency in Chrome*. 2024. (URL) (visited on 10/22/2026).

[14] Let's Encrypt. *Certificate Transparency (CT) Logs*. 2024. (URL).

[15] Stefan Santesson et al. *RFC 6960 — X.509 Internet PKI OCSP*. 2013. (URL) (visited on 10/21/2026).

[16] Mozilla. *Server Side TLS — Mozilla Security Guidelines*. 2024. (URL) (visited on 10/22/2026).

[17] Jeff Hodges, Collin Jackson, and Adam Barth. *RFC 6797 — HTTP Strict Transport Security (HSTS)*. 2012. (URL) (visited on 10/21/2026).

[18] MITRE Corporation. *CVE-2014-3566 — POODLE Vulnerability*. 2014. (URL) (visited on 10/24/2026).

[19] Acunetix. *What is POODLE Attack and How to Prevent It*. 2020. (URL) (visited on 10/24/2026).

[20] Bodo Moeller and Adam Langley. *RFC 7507 — TLS Fallback Signaling Cipher Suite Value (SCSV)*. 2015. (URL) (visited on 10/21/2026).

[21] Red Hat. *POODLE: SSLv3.0 vulnerability (CVE-2014-3566)*. 2014. (URL) (visited on 10/24/2026).